1. Принципы разработки

1. YAGNI

You Aren’t Gonna Need It / Вам это не понадобится

Если пишете **код**, то будьте уверены, что он вам **понадобится**. **Не пишите код** «на всякий случай», думая, что **он пригодится позже**.

Используйте функции git-репозиториев и учитесь фиксировать изменения правильно. После этого не бойтесь удалять лишнее.

2. DRY
Don’t Repeat Yourself / Не повторяйтесь

Дублирование кода – пустая трата времени и ресурсов. Вам придется поддерживать одну и ту же логику и тестировать код сразу в двух местах, причем если вы измените код в одном месте, его нужно будет изменить и в другом.

В большинстве случаев дублирование кода происходит из-за **незнания системы**.

3. KISS
Keep It Simple, Stupid / Будь проще

Не придумывайте к задаче более **сложного решения**, чем ей требуется.

4. Big Design Up Front
Глобальное проектирование прежде всего

Прежде чем переходить к реализации, **убедитесь, что все хорошо продумано**.

5. SOLID
Single-responsibility principle /Принцип единственной ответственности

Каждый объект, класс и метод должны отвечать только за что-то одно. Если ваш объект/класс/метод делает слишком много, вы получите спагетти-код.

Побочный эффект такого кода – проблемы с тестированием. Запутанный функционал тестировать сложно.

6. Avoid Premature Optimization
Избегайте преждевременной оптимизации

Прежде чем вы погрузитесь в детали реализации, убедитесь, что эти оптимизации действительно полезны.

Преждевременная оптимизация может привести к задержкам в коде и, следовательно, увеличит затраты времени на вывод функций на рынок.

7. Бритва Оккама

Бри́тва О́ккама (иногда ле́звие О́ккама) — методологический принцип, в кратком виде гласящий: «Не следует множить сущее без необходимости» (либо «Не следует привлекать новые сущности без крайней на то необходимости»)

-Википедия

Don’t repeat yourself

2. Шаблоны.

    Шаблоны функций

    Шаблоны классов

Классический пример на удобство использования шаблонов

```
void swap (double& a, double& b) {
    double temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    int b = 6;
    swap(a,b);                                  No matching function for call to 'swap'
    std::cout << a << b;

}
```

Следовательно, следует перегрузить функцию для другого типа данных. Но тело функции особо не изменяется:

```
void swap (double& a, double& b) {
    double temp = a;
    a = b;
    b = temp;
}
void swap (int&a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

template <typename T>

Здесь T – это параметр шаблона

```
template <typename T>
void swap (T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    int b = 6;
    swap(a,b);
    std::cout << a << b;
}
```

Несколько определений

- Обобщённое описание функции называется **шаблоном функции**. Шаблон без подстановки конкретного типа не превращается в реальный код.
- Сгенерированную конкретную функцию по шаблону называют **шаблонной функцией**.
- **Парадигму программирования**, в которой единожды описанный алгоритм может применяться для разных типов, называют **обобщённым программированием**.
- Под термином **метапрограммирование** объединяют техники написания кода, который генерирует новый код в результате исполнения.

Инстанцирование

(англ. instance — создание экземпляра чего-то) — **процесс порождения специализации**. Компилятор или разработчик где-то порождает код через подстановку параметров в шаблон, инстанцирование может быть неявным и явным.

```
template <typename T>
void swap (T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    int b = 6;
    swap(a,b);
    std::cout << a << b;
}
```

Пример неявного инстанцирования

Неявное инстанцирование не работает для неявного преобразования типов

```
template <typename T>
void swap (T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    double b = 6;
    swap(a,b);  //ошибка!
    std::cout << a << b;
}
```

Явное инстанцирование

```
template <typename Type>
Type max(Type a, Type b) {
  return (a >= b ? a : b);
}
int main () {
    int a = 5;
    double b = 6;
    int c = max<int>(a,b);
    std::cout << a << b << c;
}
```

Преобразуется к max<int>(int, int)

```
template <typename T>
void swap (T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}
int main () {
    int a = 5;
    double b = 6;
    swap<double>(a,b);
    std::cout << a << b;
}
```

Candidate function template not viable: no known conversion from 'int' to 'double &' for 1st argument

Если в функции ожидаются разные типы, можно сделать так:

```
template <typename T, typename U>
T& sum(T& a, U& b) {
    a+=b;
    return a;
}
int main () {
    int a = 8;
    double b = 5.7;
    std::cout << sum<int,double>(a, b);
}
```

Или даже так:

```
std::cout << sum(a, b);
```

Deduce. Компилятор умеет сам выводить тип

```
template <typename T, typename U>
T& sum(T& a, U& b) {
    a+=b;
    return a;
}
int main () {
    int a = 8;
    double b = 5.7;
    std::cout << sum(a, b);
}
```

При однозначном выведении типа можно его вообще не указывать либо указать частично.

```
template <typename T, typename U>
const T& min(const T& a, const U& b) {
    return (a < b)?a:b;
}
int main () {
    int a = 8;
    double b = 5.7;
    std::cout << min(a, b);
}
```

Функция min возвращает значение, и невозможно выбрать, какой тип из двух следует вернуть (T или U)?

Таким образом, при проектировании становится важным порядок расположения шаблонных аргументов. Те, которые компилятор может вывести, стоит располагать в конце списка.

Возвращаемый тип должен быть явно задан при вызове

Если тип возвращаемого значения шаблона функции является независимым параметром шаблона, он всегда **должен явно задаваться при вызове**, поскольку его невозможно вычислить на основании аргументов.

```
template<typename T, typename U>
U f(T a) {
    return a;
}
int main () {
    int a = 8;
    std::cout << f<int>(a);
}
```

No matching function for call to 'f'

Перегрузка шаблонов функций

Одноименные шаблоны функций могут перегружаться так же, как и обычные функции. Обрабатывая вызов функции в программе, компилятор должен решить, шаблон или обычная функция лучше всего подходит для данного вызова.

```
template <typename T>
const T& min(const T& a, const T& b) {
    return (a<b) ? a : b;
}
const char* min(const char* a, const char* b){
    return (std::strcmp(a,b) < 0) ? a: b;
}
double min(double x, double y) {
    return (x < y) ? x : y;
}
int main () {
    const char *s2 = "Hello";
    const char *s1 = "Good-bye";
    std::cout << min(1, 2) << std::endl;    //шаблон
    std::cout << min(1.0, 2.0) << std::endl; //double
    std::cout << min(1, 2.0) << std::endl;  //double
    std::cout << min(s1, s2) << std::endl;  //const char*
    std::cout << min<>(s1, s2) << std::endl; //шаблон
}
```

<> говорит компилятору, что вы явно хотите использовать шаблон

Приоритеты шаблонов

```
template <typename T>
void f(T);

template <typename T>
void f(T*);

template <typename T>
void f(const T*);
```

Для первого шаблона подходит любой тип. Второй шаблон более специализирован, чем первый, потому что для него подходят только типы-указатели. Третий шаблон может вызываться только для указателей на const, тогда как второй шаблон допускает любые типы указателей.

?

```
template <typename T>
void f(T& a) {
    std::cout << "T& \n";
}
template <typename T>
void f(T a) {
    std::cout << "T \n";
}


int main () {
    int x = 3;
    f(x);
    f(2);
}
```

```
template <typename T>
void f(const T& a) {
    std::cout << ”const T& \n";
}
template <typename T>
void f(T& a) {
    std::cout << "T& \n";
}


int main () {
    int x = 3;
    f(x);
    f(3);
}
```

Специализация шаблона функции

```
template <typename T>
const T& min (const T& a, const T& b) {
    return (a < b) ? a : b;
}
template<>
const char* const& min<const char*> (const char* const& a, const char* const& b) {
    std::cout << "Work again\n";
    return (strcmp(a,b)<0)?a : b;
}
const char* min(const char* a, const char* b) {
    std::cout << "Work\n";
    return (strcmp(a,b)<0)?a : b;
}

int main () {
    const char* s2 = "Hello! ";
    const char* s1 = "Hi";
    std::cout << min(s1,s2) << '\n';
    std::cout << min<>(s1,s2) << '\n';
}
```

Лучше использовать перегрузку

Перегрузка и специализация шаблона функции – это разные вещи

```
template <typename T, typename U>
void f(T, U) {
    std::cout << 1;
}
template <>
void f(int, int) {
    std::cout << 3;
}
template <typename T>
void f(T, T) {
    std::cout << 2;
}


int main () {
    f(0,0); //2
}
```

```
template <typename T, typename U>
void f(T, U) {
    std::cout << 1;
}
template <typename T>
void f(T, T) {
    std::cout << 2;
}
template <>
void f(int, int) {
    std::cout << 3;
}


int main () {
    f(0,0);   //3
}
```

Любая частичная специализация шаблона функции будет восприниматься как перегрузка шаблона

Выводы

1. Шаблонные функции не имеют реального кода, пока не появится их явный вызов с инстанцированием. Всё написанное можно воспринимать как «псевдокод».
2. Поэтому компилятору необходимо уметь точно определять тип возвращаемого параметра, остальные типы он может иногда вывести сам.
3. Если есть нешаблонная функция, которая будет оптимально подходить по передаваемым параметрам, выберется именно она, если не указать явное требование использовать шаблон.
4. Можно использовать специализацию шаблонной функции, но лучше пользоваться простой перегрузкой.
5. Специализация пишется сразу после шаблона.

Шаблон класса

```
template <typename T>
class vector {
    T* arr;
    size_t sz;
    size_t cap;
};
```

При объявлении генерируются разные классы для разных типов

```
int main () {
    vector<int> A;
    vector<char> B;
    vector<double> C = A;  //Будет ошибка
}
```

Нетиповой параметр шаблона (у функций также может использоваться)

Нетиповой параметр шаблона должен быть целочисленным значением, известным на стадии компиляции. Начиная с C++20 не только целочисленным.

```
template <typename T, size_t N>
class array {
    T arr[N];
};


int main () {
    array<int, 20> A;
    array<char, 100> B;
}
```

Аргументы шаблона по умолчанию (у функций также разрешены)

```
template<typename T = int, size_t N = 100> // Значения по умолчанию
class Stack {
    T data[N]; // фиксированная емкость N
    size_t count;
    void push(const T& d);
    // и т. д.
};
```

Параметризация типов на примере

```
template <size_t N, size_t M, typename T = double>
class Matrix {
    T matrix[N][M];
};

template <size_t N, typename T = double>
using SMatrix = Matrix<N,N,T>;

template <size_t N, size_t M, size_t K, typename T = double>
Matrix<M,N,T> operator* (const Matrix<M,K,T>& left, const Matrix<K,N,T>& right);
```

Специализация шаблона класса

```
template <typename T, typename U>
class C {
public:
    void f (){ std::cout << "Primary\n";}
};

template<typename U>
class C <int, U> {
public:
    void f(){ std::cout << "T = int\n"; }
};

template<typename T>
class C<T, double> {
public:
    void f() { std::cout << "U = double\n"; }
};
```

```
template<typename T, typename U>
class C<T*, U> {
public:
    void f() { std::cout << "T* used\n"; }
};

template<typename T, typename U>
class C<T, U*> {
public:
    void f() { std::cout << "U* used\n"; }
};

template<typename T, typename U>
class C<T*, U*> {
public:
    void f() { std::cout << "T* and U*\n";}
};
```

```
int main () {
    C<float,int>().f();
    C<int,float>().f();
    C<float,double>().f();
```

```
    C<float,float>().f();
    C<float*,float>().f();
    C<float,float*>().f();
}
```

Частичная специализация методов классов запрещена

```
template <typename T, typename U>
struct C {
    void f(T a, U b);
};

template <typename T, typename U>
void C<T, U>:: f(T a, U b){
    a = b;
}

template <typename T>
void C<T, int>:: f(T a, int b){
    a = b;
}

template <>
void C<int, int>:: f(int a, int b){
    a = b;
}
```

Nested name specifier 'C<T, int>::' for declaration does not refer into a class, class template or class template partial specialization

2 Use of undeclared identifier 'a'

Шаблоны как параметры шаблонов

```
template<typename T>
class Array {
    T *data;
    size_t capacity;
    size_t count;
public:
    Array(): count(0), data(new T[capacity = 10]) { }
    ~Array() { delete [] data; }
    T* begin() { return data; }
    T* end() { return data + count;}
};

template<typename T, template<typename> typename Seq>
class Container {
    Seq<T> seq;
public:
    T* begin() { return seq.begin();}
    T* end() { return seq.end(); }
};

int main () {
    Container<int, Array> container;
    int* p = container.begin();
    while(p != container.end()) {
        std::cout << *p++ ;
    }
}
```

Вложенные шаблоны

```
template<typename T>
class Complex {
    T var;
public:
    Complex(T var):var(var){}
    template <typename X> Complex(const Complex<X>& c);
};


template<typename T>
template<typename X>
Complex<T>:: Complex(const Complex<X>& c) {/*Тело определения... */}


int main () {
    Complex<int> c(6);
    Complex<double> c1(c);
}
```

Друзья шаблонных классов.
Вариант 1. Определение внутри класса

```
template<class INF>
class MyStack {
    using Node = ListNode<INF, MyStack<INF>>;
    Node* top;
    //...

friend std::ostream& operator<<(std::ostream& os, const MyStack<INF>& st) {
    Node* p = st.top;
    //...
    return os;
}
};
```

Здесь оператор является простой функцией. При каждой специализации класса создаётся новая версия оператора, которая получает аргумент с типом текущей специализации.

Друзья шаблонных классов.
Вариант 2. Определение вне класса

Когда дружественная функция реализована вне класса, она в качестве параметра должна принимать наш шаблонный класс. Таким образом она будет специализирована для нашего класса. Так как это специализация, следует выполнить предопределение функции.

```
template<typename T> class MyStack; // pre-declare the template class itself
template<typename T> std::ostream& operator<< (std::ostream& o, const MyStack<T>& x);

template<class INF>
class MyStack {
    using Node = ListNode<INF, MyStack<INF>>;
    Node* top;
    //...
    friend std::ostream& operator<< <INF>(std::ostream& os, const MyStack<INF>& st);
};

template<class INF>
std::ostream& operator<<(std::ostream& os, const MyStack<INF>& st) {
    ListNode<INF, MyStack<INF>>* p = st.top;
    //...
    return os;
}
```

Друзья шаблонных классов.
Вариант 3. Все экземпляры шаблона объявляются дружественными

```
template<class INF>
class MyStack {
    using Node = ListNode<INF, MyStack<INF>>;
    Node* top;
public:
    //...
    template <typename U>
    friend std::ostream& operator<< (std::ostream& os, const MyStack<U>& st);
};

template<class INF>
std::ostream& operator<<(std::ostream& os, const MyStack<INF>& st) {
    //...
}
```

Компиляция проекта при наличии шаблонов

Принцип раздельной компиляции у проектов, использующих шаблоны, неприменим. То есть нельзя разделить объявление и определение шаблонных классов (функций) в соответствующие файлы hpp и cpp.

Почему?

Шаблоны не являются полноценным кодом, только на этапе компиляции формируется код на основании определения параметров шаблона (инстанцирование).

Но каждый cpp файл после препроцессинга компилируется независимо от других как отдельная единица трансляции в объектный файл. Таким образом, после получения объектного файла реального определения функции в нём нет, потому как непонятно, чем её инстанцировать.

Компиляция шаблона

На этапе линковки происходит поиск соответствующей функции, но нигде подходящей функции нет, будет ошибка.

```
#include "Test.hpp"
template <typename T>
T f (T a, T& b){
    b+=a;
    return b;
}
```

```
template <typename T>
T f (T a, T& b);
template <typename T>
T f (T a, T& b){
    b+=a;
    return b;}
```

Test.o

```
template <typename T>
T f (T a, T& b);
```

```
#include "Test.hpp"
int main () {
    int a = 6;
    int b = 3;
    f<int>(a,b);
}
```

```
int f (int a, int & b);
int main () {
    int a = 6;
    int b = 3;
    f<int>(a,b);
}
```

main.o

```
Undefined symbols
  "int f<int>(int, int&)", referenced
from: _main in main.o
```

Решение #1

В файле cpp прописывать все возможные реализации, произвести, так называемую, ручную генерацию специализаций

```
#include "Test.hpp»

template int f<int>(int,int&);
template double f<double>(double,double&);

template <typename T>

T f (T a, T& b){
    b+=a;
    return b;
}
```

Решение #2

Использовать tpp. И в h-файле написать в конце #include “Test.tpp”

```
template <typename T>
T f (T a, T& b);

#include “Test.tpp"
```

Как вы думаете, что будет, если вместо tpp использовать zzz? ☺

По-настоящему, разницы нет, какой файл включать. Препроцессор раскрывает include, и компилятор получает h-файл с объявлением и определением (см. Решение #3)

Решение #3

Объявление и определение прописывать в одном заголовочном файле hpp (см. cppreference.com)

Test.hpp:

```
template <typename T>
T f (T a, T& b);

template <typename T>
T f (T a, T& b){
b+=a;
return b;
}
```

При этом пустой cpp-файл, имеющий единственную строку *“#include Test.hpp”*, очевидно, не нужен, но если очень хочется, см. дальше

Особенности специализации

Специализированный метод класса или специализированная шаблонная функция – это полноценная функция с вполне себе полноценным определением, она УЖЕ инстанцирована. Если специализация выполнена в h-файле, но при этом есть ещё и cpp, пусть даже пустой, то при компиляции получается на выходе единица трансляции с определением функции.

Таким образом, при очередном включении этого h-файла в другую единицу трансляции, там также сформируется определение этой же функции. То есть у одной функции получится множество определений, что противоречит принципу One Definition Rule, ODR.

Решение #1

Не использовать cpp, а в hpp прописать защиту от повторных включений.

Решение #2

Специализацию функций выносить в cpp. В заголовочном файле останется объявление (объявлений, как известно, может быть очень много), а реализация будет одна.

Решение #3

Использовать inline, таким образом вписывая реализацию функции непосредственно в код, что защищает от ошибки повторного определения.